Distributed Configuration Updater: Design and Implementation of a CRDT-Based Settings Replication System

Gabriele Da Re
Dept. Mathematics
Università degli Studi di Padova
Padova, Italy
gabriele.dare00@gmail.com

***Abstract*—This paper describes the architecture, implementation and decisions made during the creation of a configuration synchronization system for server clusters. The system synchronises Key-Value (KV) entries across nodes through: Last-Writer-Wins (LWW), Conflict-free Replicated Data Type (CRDT), gRPC transport protocol and a Hybrid Logical Clock (HLC) for causal ordering. We explain the modules design decisions and the problems encountered/mitigated during development such as: Clock-skew, crash persistence handling, tombstones management and the anti-entropy approach and lifecycle enforcement between initialisation and runtime phases.**

***Index Terms*—CRDT, distributed systems, data consistency, hybrid logical clocks, gRPC, Go, key-value store**

I. Introduction

Configuration management across a server cluster is a common operational challenge. Changes to settings such as feature flags, service endpoints, parameters must propagate to every node without a coordinator becoming the single failure point.

Approaches such as etcd, Consul rely on a leader node or a distributed database. These provide linearisability at the cost of availability (Consistency and Partition tolerance part of the CAP theorem [2]; ACID principle): since RAFT [1] based, when the leader or the majority ($N/2 + 1$ nodes) is unreachable, there is no working system (not always Available; CAP [2]). For configuration data, human operators can intervene on conflicts. Moreover, having stale data in some circumstances is relatively low risk. Therfore, the trade-off favours Availability and Partition tolerance (AP [2]) at the cost of strong Consistency.

Conflict-free Replicated Data Types (CRDTs) [3] offer an alternative: every node can accept writes locally, the CRDT, a join-semilattice under merge, guarantees that any two nodes with the same set of updates will converge to the same state without inter-node coordination. This paper describes a practical implementation of a CRDT based configuration store targeting small, fixed-membership clusters where $O(100)$ configuration keys are replicated across $O(10)$ nodes. This limit can be pushed up to $O(1000)$ keys and $O(50)$ nodes if necessary, based upon limits imposed by the full snapshot sync implementation (Section VII).

The implementation is written in Go and uses gRPC for inter-node communication. The remainder of this paper is structured as follows. Section II presents the overall system architecture. Section III describes the CRDT data model. Section IV covers the hybrid logical clock. Section V explains the actor-based concurrency model. Section VI details the networking module. Section VII discusses the periodic state reconciliation. Section VIII describes crash-safe persistence. Section IX presents tombstone garbage collection. Section X covers lifecycle and error handling. Section XI outlines the testing strategy. Section XIII presents the conclusions.

II. System Architecture

The code is structured as five Go packages with a strict dependency hierarchy, shown in Table I.

Table I
SYSTEM MODULE BREAKDOWN

Package	Responsibility
`types`	Shared domain types: HLC, SettingEntry, Snapshot
`crdt`	LWW CRDT, state machine, persistence
`logic`	Local settings file watcher and writer
`network`	gRPC server, streaming client, proto conversion
`controller`	Wires all packages, lifecycle and backgrounds tasks
`main`	CLI entry-point

The data flows through the system following four paths, wired up in `controller.Run`:

1) **Local write**: the `logic` watcher detects file changes on write and pushes a `SettingEntry` to `crdt.NotifyLocal`, which queues it in local channel of the CRDT actor. The CRDT ticks its clock, timestamps the entry and merges it, persists state, and calls `OnBroadcast` delegating the broadcast to the network layer.
2) **Remote write**: a peer to which the node is subscribed streams an update, `network.Client` calls `crdt.NotifyRemote`. The CRDT updates its clock,

merges, persists, and calls `OnFileSync` that delegates the `logic` layer to write the change locally.

3) **Anti-entropy**: a ticker in the controller calls `syncAllPeers`, which sends the local snapshot to each peer via a Sync RPC which returns (if any) any newer entries through `NotifyRemote`.
4) **Tombstone GC**: another separate ticker calls `crdt.PurgeTombstones`, which removes stale deleted entries from the in-memory state map via the CRDT actor model.

The controller is the only package that depends on all the others, preventing import cycles.

III. CRDT Data Model

A. Data Model

The data structure is an **LWW-register** [3] map keyed by setting name. Per Shapiro, each register is any local data type X paired with its timestamp from the $assign$ function. In our case X is a $(key, value, Deleted)$ triplet, and the timestamp is managed through an HLC (section IV). The key is stored alongside its value so the map can be rebuilt on the destination node, whilst the `Deleted` flag deals with deleted entries on the setting file, enabling the local module to drop locally persisted settings.

```
type SettingEntry struct {
    Key     string
    Value   string
    Clock   HLC
    Deleted bool
}
```

B. Merge Semantics

Two nodes merge by iterating the local map against the remote one and checking each key: if the key exists locally with a higher clock is kept unaltered, else it is created or overwritten. On equal clocks, the lexicographically larger NodeID is used as tiebreaker, ensuring a deterministic total order.

```
func (c *CRDT) merge(incoming SettingEntry) bool {
    existing, ok := c.state[incoming.Key]
    if ok && !existing.Clock.Before(incoming.Clock)
        {
        return false
    }
    c.state[incoming.Key] = incoming
    return true
}
```

C. Tombstone Semantics

Deletions are managed by setting the `Deleted` flag to `true` and stamping a fresh clock on the associated triplet. Tombstones undergo the same LWW merge as the payload and can be displaced by a later write. This avoids the anomaly where a deletion arrives after a concurrent write and erroneously removes the newer value.

D. CvRDT

The implementation is fundamentally a state-based CRDT: a timed `Sync` provides anti-entropy reconciliation and each local state change broadcasts a `SettingEntry` to peers (operation-based style). Because LWW merge is commutative and associative, a set of updates can be applied regardless of the order to obtain the same result, and idempotent, makes a merge of $B \subseteq A$ a no-op. Therfore, each broadcast is algebraically equivalent to a partial state merge. Convergence is guaranteed entirely by the state-based anti-entropy `sync` cycle, the broadcast is a best-effort optimisation.

E. CvRDT proof

a) State forms a join-semilattice.: Define the replica state as $S = \text{Map}[String \to SettingEntry]$.
Impose a partial order $\forall k \in Keys$

$$S_1 \leq S_2 \iff \forall k : S_1[k].Clock \leq S_2[k].Clock \tag{1}$$

where `Before` defines the strict order $<$ on clocks (Definition IV), and $\leq$ is its reflexive closure: $a \leq b \iff a < b \vee a = b$. The least upper bound per key is

$$(S_1 \sqcup S_2)[k] = \arg\max_{Clock}\big(S_1[k],\ S_2[k]\big) \tag{2}$$

which is exactly `merge()`.

b) Merge implements the LUB.: Let A, B, C be `SettingEntry` values for the same key.

- **Commutative:** $\text{merge}(A, B) = \text{merge}(B, A)$, since $\max(Clock_A, Clock_B)$ is symmetric.
- **Idempotent:** $\text{merge}(A, A) = A$, since $\max(T, T) = T$.
- **Associative:** $\text{merge}(A, \text{merge}(B, C)) = \text{merge}(\text{merge}(A, B), C)$, since $\max$ is associative.

c) Monotonicity.: Every merge moves state upward in the lattice:

$$S \leq \text{merge}(S,\ e), \tag{3}$$

since `merge` replaces an entry only when the incoming clock is strictly greater, and never decreases a clock value.

d) Liveness.: `syncAllPeers` runs periodically exchanging full snapshots. After a network partition, reconnection triggers `Sync`, delivering all missing updates. Hence every update eventually reaches every replica.

e) Conclusion.: By the Eventual Consistency theorem for CvRDTs [3]: any two replicas that have received the same set of updates hold identical maps, regardless of delivery order.

IV. Hybrid Logical Clock

A. Motivation

A Lamport clock [4] provides causal ordering but loses the physical-time locality property: events that happen close together in real time may receive widely separated clock values if logical increments accumulate. Hybrid Logical Clocks [5] integrate the Lamport clock with a wall time, preserving the benefits of both.

B. Clock Structure

Each node maintains an HLC as a triple $(wall, logical, nodeID)$. The wall field stores Unix nanoseconds, the logical field is a 32-bit logical counter, the NodeID is a UUID string used as tiebreaker (Section III-B). This ensures `Before()` commutativity since $\forall n_i, n_j \in Nodes,\ i \neq j : Id(n_i) \neq Id(n_j)$

C. Tick and Update Rules

On a local event (*tick*), the clock advances (ℓ_l, ℓ_r for $logical_{local}, logical_{received}$, w_l, w_r for $wall_{local}, wall_{received}$):

$$wall' = \max(w_l,\ now),$$
$$logical' = \begin{cases} logical + 1 & wall' = w_l \\ 0 & otherwise \end{cases} \tag{4}$$

On receiving a remote clock (*update*), the canonical Kulkarni–Demirbas rule [5] is applied:

$$wall' = \max(w_l,\ w_r,\ now) \tag{5}$$

The logical counter then captures concurrency at the chosen wall time:

$$logical' = \begin{cases} \max(\ell_l, \ell_r) + 1 & w' = w_l = w_r \\ \ell_r + 1 & w' = w_r \neq w_l \\ 0 & otherwise \end{cases} \tag{6}$$

D. Clock Skew Handling

The system assumes NTP-disciplined clocks with bounded skew ($\lesssim$10ms on LAN, $\lesssim$100ms on WAN. CockroachDB's empirically chosen tolerance is 500ms [6]). Under this assumption HLC self-corrects: when a node physical clock falls behind the cluster wall time, the logical counter increments, preserving causality without advancing the wall.

An initial design rejected incoming entries whose `WallTime` exceeded the receiver's physical clock by more than a fixed threshold T:

$$\textit{received.WallTime} - \textit{now} > T \implies \text{reject} \tag{7}$$

This check merges the HLC wall with the sender's physical clock. Consider a cluster at physical time 0, a fast node B at $+0.9s$, and a slow node A at $-0.9s$ (so $|t_A - t_B| = 1.8s > T$):

1) Cluster accepts B: $\delta = 0.9s < T$. HLC wall bumped to $+0.9s$.
2) Cluster sends its own entries to A. Those entries carry `WallTime` $= +0.9s$.
3) A computes $0.9 - (-0.9) = 1.8s > T$ and rejects, despite A and the cluster being only $0.9s$ apart physically.

This produces a **self-healing partition**: A stops receiving updates despite full network connectivity. The partition resolves automatically once NTP corrects A's clock past the acceptance threshold ($now_A > HLC_{\text{cluster}} - T$), after which the next anti-entropy cycle delivers all missed entries and convergence resumes. However, NTP slews clocks gradually (does not step unless the offset exceeds $\approx 128ms$ by default), so the partition may persist for minutes to hours. If both A and the cluster are stuck with bad NTP indefinitely the partition does not heal at all.

The CRDT merge function remains correct so safety is preserved; the failure point is at the delivery layer, violating the liveness precondition of Strong Eventual Consistency [3].

Note that the check could have been maintained if: (i) the check was performed on the sender's physical clock rather than the HLC wall, so each node judges independently; (ii) the threshold was set to a greater value such as 30s, to detect real skew problems while avoiding triggers on NTP jitter. In the current implementation, skew check is removed, protection is delegated entirely to NTP and operational monitoring.

V. Actor Model

All mutations to the CRDT state map are serialised through a single actor goroutine running a `select` loop over five typed channels:

```
type CRDT struct {
    state      map[string]SettingEntry
    localCh    chan SettingEntry
    remoteCh   chan SettingEntry
    queryCh    chan queryRequest
    snapshotCh chan snapshotRequest
    gcCh       chan int64
}
```

No mutex is required: only the actor goroutine accesses channels and CRDT state ensuring operation atomicity. External callers (`NotifyLocal`, `NotifyRemote`, `Snapshot`, `queryRequest`, `PurgeTombstones`) communicate exclusively through channels. Snapshot and query requests are buffered synchronous response channels thus blocking until the request is serviced.

Callbacks (`OnBroadcast`, `OnFileSync`) are invoked synchronously within the actor loop. The `OnFileSync` callback dispatches disk writes on a separate goroutine to prevent blocking the actor loop on I/O.

VI. Networking Module

A. gRPC Service

The inter-node protocol exposes two RPCs over gRPC:

- **SubscribeStateUpdates** (server-streaming): the caller receives `ServerStateUpdate` messages through a persistent stream every time the server calls `Broadcast`. This ensures real-time propagation of local edits.
- **Sync** (unary): the caller sends its local snapshot, the server updates its entries and responds with local server side entries that are newer. Used for anti-entropy reconciliation.

The server maintains a map of subscriber channels protected by a mutex. `Broadcast` under read-lock retrieves the subscribed peers channels, then sends the update to each channel in a non-blocking manner. Slow subscribers drop updates relying on the next anti-entropy pass to recover.

B. Abstraction Boundary

A design decision was taken such that all protobuf conversion (`ToProto`, `FromProto`, `SnapshotToProto`) is confined in the `network` package. The `Client` methods accept and return domain types (`types.Snapshot`, `types.SettingEntry`). Conversion to and from wire format is performed internally.

This was not in the original design: an earlier iteration of `Client.Sync` and `Client.Broadcast` accepted `[]*userpb.SettingEntry` and `[]*userpb.ServerStateUpdate`, thus requiring the controller to perform the conversion before each call. Refactor moved conversion dependency into network package, making the transport layer a true black box.

C. Reconnection and Keepalives

`Subscribe` method in `Client` wraps the streaming RPC in a retry loop: on any error, including server restart or network interruption, it waits three seconds and tries to re-establish the stream, performing a Sync call first to replay any entries missed during the outage. gRPC keepalive parameters are configured to detect dead connections.

VII. Anti-Entropy Reconciliation

Broadcast over gRPC is best-effort: a slow, offline or reconnecting subscriber may miss updates. A (state-based) CRDT that relies solely on broadcast for propagation is not eventually consistent in presence of message losses, nodes that miss a broadcast permanently diverge.

Production CRDT deployments (Riak, Dynamo [7]) address this with a background anti-entropy mechanism: periodic full-state comparison identifies and repairs divergence independently of the broadcast path (Riak AAE).

This system implements a lightweight variant: `syncAllPeers` is called periodically by the controller; this sends a local snapshot to each peer via the Sync RPC, updating peer entries and also local entries on returned values. This is sufficient for eventual consistency: even if every broadcast is dropped, the state will converge within one anti-entropy interval.

There is an unimplemented enhancement regarding exchanges of redundant KV pairs: it can happen that 2 nodes send a Sync request at the same instant; each node takes a snapshot of local state and sends it, then upon receiving the snapshot both nodes update their local entries and send the updates they consider necessary to the other, which has already applied them through the received snapshot. This is solvable but out of scope for this project (e.g. Jitter, follower/leader Sync, Merkle tree, Delta state, etc.).

```
func (ctrl *Controller) syncAllPeers(ctx context.
    Context) {
    snap := ctrl.crdt.Snapshot()
    for _, client := range ctrl.clients {
        go client.Sync(ctx, snap, func(e
            SettingEntry) {
            ctrl.crdt.NotifyRemote(e)
        })
    }
}
```

A 60-second interval was chosen as default: bounding divergence to one minute in worst-case broadcast loss and generating negligible network traffic for the cluster size targeted.

VIII. Crash-Safe Persistence

After each merge, CRDT serialises its state to `crdt_state.json` found in the CRDT working directory. The naive approach using `os.WriteFile` is not crash-safe: if the process is killed after `WriteFile` had truncated the file and before the kernel had flushed all pages to disk, the node restarts with a corrupted state file.

The correct pattern we used is the atomic rename:

```
func atomicWriteFile(path string, data []byte) error
    {
    tmp, err := os.CreateTemp(filepath.Dir(path), ".
        tmp_*")
    // ... write data to tmp, call tmp.Sync(), tmp.
        Close() ...
    return os.Rename(tmp.Name(), path)
}
```

Three properties make this safe:

1) The temporary file is in the same directory of the target, thus the rename is within a single filesystem so, atomic at VFS layer on POSIX systems.
2) `tmp.Sync()` flushes file data to storage before the rename, preventing data from remaining only in the page cache during power-loss.
3) `os.Rename` is atomic on POSIX: any reader will see either the old file or the new file, never a partially-written intermediate state.

If the process crashes before the rename, the temporary file is left in the directory and cleaned up during the next `crdt.Init()` call, the original file remains intact.

On any restart, `crdt.Reconcile` is called once from `controller.Run()`, fully restoring local file integrity. This is a limiting factor since on an already initialized node, any edits made on the settings file while the node is offline will be discarded in favour of the last known state.

IX. Garbage Collection

A. Problem

Deleted entries persist in CRDT state file indefinitely; this also happens for **temporary files** generated by `atomicWriteFile` upon system crash. In systems with frequent delete-recreate cycles over large KV stores, tombstone accumulation brings two problems: memory leak over time, slower `Sync` and `crdt.Run` since larger snapshots are required.

B. Init cleanup

With regard to orphaned tmp files from `atomicWriteFile`, the best solution in this case was also the easiest: removal of all pattern-matched tmp files on the `crdt.Init` call.

C. TTL-Based Approach

The system also implements a TTL-based garbage collector. A ticker in the controller calls `crdt.PurgeTombstones(ttl)` passing the TTL. The CRDT actor then processes the request from `gcCh` channel monitored in its Run loop, thus maintaining the single-writer on `c.state`, removing tombstones with wallTime older than `now - ttl`.

```
func (c *CRDT) purgeTombstones(ttl int64) {
    diff := time.Now().UnixNano() - ttl
    purged := 0
    for key, entry := range c.state {
        if entry.Deleted && entry.Clock.WallTime <
            diff {
            delete(c.state, key)
            purged++
        }
    }
    if purged > 0 {
        c.saveState()
        log.Printf("tombstone GC: purged %d entries
            ", purged)
    }
}
```

The default TTL is 14 days, twice the intended maximum node downtime after which an operator would investigate. This ensures that tombstones have been observed and applied by all healthy nodes via anti-entropy before they are removed. After a week of downtime a node typically requires re-initialization.

D. Safety Considerations

The TTL approach has one known failure point: if a node is offline for longer than the TTL and holds a non-deleted entry for a key tombstoned and GC'd on all other nodes, the node will republish the entry, so the deletion will appear to be reversed. This is solved by forcing the node to re-initialize with empty settings if, on startup, `time.Now() - lastShutdownTime > TTL`. Reinit through `fixNodeState` discards any unsynced local CRDT state but cluster state is fully restored via anti-entropy. The loss is bounded to changes made only on the offline node's disk, never propagated before it went offline. This is verified through `TestStaleTTLReinit` in `system_test.go`.

There are also 2 known limitations called the sputtering cycle, determined by Sync requests taking a CRDT state snapshot before a GC request is serviced in the CRDT actor. This makes the Sync cycle bring along GC'd entries, creating the problem. This is mitigated through wire-layer filtering in the `Sync` handler. Before including a local entry in the response or applying an incoming remote entry, the handler skips any entry where `Deleted = true` and `now - wallTime > TTL`. Filtering is applied in both directions preventing resurrection of tombstones regardless of GC cycles timing skew or snapshot-before-GC ordering between nodes.

X. Lifecycle and Error Handling

A. Init / New Separation

The CRDT package exposes three functions:

- `New(workDir)` instantiates a `*CRDT` value with buffered channels. Pure allocation, no I/O, no error.
- `Init()` loads an existing node: cleans up any orphaned atomic-write tmp files, reads `node_id`, and deserialises `crdt_state.json`. Returns an error if the working directory has not been initialised (*"run init first"*).
- `InitNew(entries)` bootstraps a fresh node: creates the working directory, writes `node_id` if absent, stamps all supplied entries with a fresh HLC tick, and persists the resulting state. Used both at first-time setup and when forcing a clean slate.

The controller decides which path to take via `fixNodeState`:

```
func (ctrl *Controller) fixNodeState(lastAlive int64
    ) error {
    if lastAlive != -1 &&
        time.Now().UnixNano()-lastAlive > ctrl.
            tombstoneTTL {
        return ctrl.crdt.InitNew(types.Settings{})
    }
    return ctrl.crdt.Init()
}
```

The input is `lastAliveTime()`, defined as $\max(last\_shutdown, last\_heartbeat)$. `last_shutdown` is written only by the deferred `saveShutdownTime()` on graceful exit. `last_heartbeat` is (by default) written every `TombstoneTTL/4` with a dedicated goroutine in `Run`, providing a proof-of-life signal even when the node never reaches a graceful shutdown. Without the heartbeat the reinit suffers two symmetric failures: a node that crashed before any graceful shutdown would always take the `Init()` branch on restart, even after weeks offline (cluster pollution). A node that ran for years past its last graceful shutdown would always take the `InitNew()` branch on a brief crash. Both files return `-1` on the first startup after `InitNew`, going through `Init()` on the freshly-written state.

B. Error Propagation Principle

Functions in library packages (`crdt`, `network`) return errors rather than calling `log.Fatal` or `os.Exit` directly. `log.Fatal` belongs only at the application boundary (`main`) where there is sufficient context to decide that an error is truly unrecoverable. This makes every package independently testable without the test process exiting.

XI. Testing Strategy

A. Clock Injection

The `HLC` struct accepts a `Clock` interface via parametrized options at construction time. Tests inject `MockClock` to control wallTime advancement deterministically, covering edge cases such as backward clock steps, simultaneous ticks, and the case where system clock overtakes both local and received wallTime, resetting logical counter to zero.

B. Network Tests with bufconn

The `network` package is tested with `google.golang.org/grpc/test/bufconn`, an in-memory listener that presents a real gRPC connection without opening any network socket. This allows tests to exercise the full gRPC stack without port allocation or network related problems.

C. Coverage

Line coverage targets were established and maintained: `types` at 100%, `crdt` at approximately 90%, `logic` at approximately 90%, `network` at approximately 91%, and `controller` at approximately 92%. The remaining uncovered branches are structurally untestable without process-level mocking, `json.Marshal` on `map[string]string` (the error path never fires in the Go standard library), and the `fsnotify` closed-channel path (fires only after the watcher goroutine has already exited). Unit tests `TestMerge_SameWallTimeTiebreakByLogical` and `TestMerge_SameWallAndLogicalTiebreakByNodeID` verify the logical-then-nodeID tiebreak defined in Section III, covering the concurrent-write scenario that system-level tests cannot exercise.

D. System Tests

System tests run against a live three-node cluster deployed with Docker Compose.

a) Infrastructure.: Logic helpers are built such `writeToNode` injects entries via the `Sync` RPC. A `wallOffset` parameter shifts the injected HLC wall time, enabling conflict-resolution scenarios without touching the system clock. Convergence is verified by `waitKeyValue`, `waitKeyDeleted`, and `waitKeyPurged`, each polling all nodes until the expected condition holds or a deadline is reached. Docker helpers (`stopNode`, `startNode`, `waitForNodeUp`) drive container lifecycle for fault-tolerance tests. The compose environment overrides `tombstoneTTL` to $10s$ and `gcInterval` to $3s$ so that GC and TTL reinit tests complete in under a minute.

b) Basic write propagation.: `TestInitialStateConverges` verifies that node1's seed entry (`da=ddd`) reaches all peers through the startup anti-entropy pass. `TestWritePropagatesAcrossNodes` injects a distinct key on each of the three nodes and asserts cluster-wide convergence, covering every origination point in a single test. Concurrent write tests are performed at unit level since the actor model serializes the writes at system level.

c) Local file-watcher path.: `TestFileWatcherPropagates` edits node1's `settings.json` from outside the controller (via `docker exec`), bypassing the gRPC `Sync` ingress that every other test uses. This closes the end-to-end loop on the production write path that the unit-level `TestRun_LocalFileChangePropagatesToCRDT` only covers in-process.

d) LWW conflict resolution.: `TestHLCConflictResolution` writes the same key to two nodes with intentionally different HLC wall times (loser at $-2s$, winner at $+500ms$) and asserts that the cluster converges to the winner. `TestOverwriteConverges` confirms that a later write (higher HLC) on a key that has already converged cluster-wide displaces the earlier value. `TestOlderWriteDoesNotOverwrite` sends a stale write ($-5s$ wall offset) to a node that already holds a newer value, waits for any incorrect propagation to surface then asserts the newer value over the cluster.

e) Tombstone lifecycle.: `TestTombstonePropagates` writes a key and waits for convergence then deletes it via a tombstone on node1 and verifies cluster convergence. `TestTombstoneResurrection` writes a key, deletes it, then writes the same key again with a $+2s$ HLC offset. The cluster must accept the resurrection since the new HLC beats the tombstone's clock. This is a guard against tombstones permanently blocking the re-use of a key. `TestTombstoneGC` extends `TestTombstonePropagates` by waiting for the tombstone's wall time to age past TTL and the GC loop to fire, confirming the entry is absent from `Sync` responses on every node.

f) Fault tolerance.: `TestNodeRestartConverges` stops node1, writes a key on node2 while it is offline, restarts node1 and asserts that anti-entropy delivers the missed entry to node1. `TestSIGKILLRecovery` proves an ungraceful crash leaves `crdt_state.json` intact. After a write converges, two peers are stopped so node1 on restart cannot sync, then is killed by `SIGKILL`. The normal refresh of `last_shutdown` is bypassed because of SIGKILL. The heartbeat ticker keeps `last_heartbeat` fresh so `lastAliveTime` on restart resolves on a recent timestamp and `fixNodeState` takes the `Init()` branch. Node1 must still hold the pre-crash key, proving that persistence held under a crash. `TestStaleTTLReinit` goes through the forced-reinit path: after stopping node1, overwrites both `last_shutdown` and `last_heartbeat` with a timestamp $11s$ in the past (exceeding the $10s$ TTL), injects a future-clock entry into node1 on-disk CRDT state (a winning entry) and restarts. If `fixNodeState` correctly calls `InitNew`, the injected entry is wiped before any propagation and node2's value is spread cluster-wide, if reinit is skipped the injected entry's higher HLC beats node2 and the test fails.

Table II summarises all system tests.

XII. AI USAGE AND APPLICATION

XIII. CONCLUSION

This paper presented a production-quality CRDT-based configuration synchronisation system. The core LWW CRDT provides convergence under concurrent writes and network partitions. The actor concurrency model eliminates lock contention. Anti-entropy reconciliation bounds divergence to Sync interval regardless of broadcast reliability. Atomic persistence through

Table II
SYSTEM TEST SUITE

Test	Property verified
`TestInitialStateConverges`	Startup anti-entropy delivers seed data
`TestWritePropagatesAcrossNodes`	Write from any node reaches all peers
`TestFileWatcherPropagates`	External `settings.json` edit propagates via fsnotify
`TestHLCConflictResolution`	Higher HLC wins conflict
`TestOverwriteConverges`	Later write displaces earlier value
`TestOlderWriteDoesNotOverwrite`	Stale write does not overwrite newer
`TestTombstonePropagates`	Delete tombstone reaches all nodes
`TestTombstoneResurrection`	Newer write defeats existing tombstone
`TestTombstoneGC`	Tombstone purged after TTL expiry
`TestNodeRestartConverges`	Offline node catches missed updates
`TestSIGKILLRecovery`	State survives ungraceful crash
`TestStaleTTLReinit`	Long-offline node forces clean reinit

`os.Rename` prevents corruption on ungraceful shutdown. Tombstone GC fixes CRDT state growth for long-running deployments.

Several design decisions evolved during implementation. The protobuf abstraction boundary was moved into the `network` package, removing transport-layer knowledge from the controller. Error propagation got modified to return `error` from library code and keep `log.Fatal` in the application entry-point. Forceful init was added for nodes rejoining the cluster after more than TTL elapsed since last shutdown. A full reconciliation cycle was added upon start to make sure ungraceful shutdowns did not produce stale entries.

The system does not address gossip-based broadcast efficiency, or metrics observability, all known gaps appropriate for future work in production deployments at larger scale.

REFERENCES

[1] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annual Technical Conf. (USENIX ATC)*, Philadelphia, PA, Jun. 2014, pp. 305–319.

[2] S. Gilbert and N. A. Lynch, "Perspectives on the CAP theorem," *Computer*, vol. 45, no. 2, pp. 30–36, Feb. 2012.

[3] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "A comprehensive study of convergent and commutative replicated data types," INRIA, Tech. Rep. RR-7506, 2011.

[4] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978.

[5] S. S. Kulkarni and M. Demirbas, "Logical physical clocks and consistent snapshots in globally distributed databases," in *Proc. 18th Int. Conf. Principles of Distributed Systems (OPODIS)*, Cortina d'Ampezzo, Italy, Dec. 2014.

[6] CockroachDB Engineering, "Living without atomic clocks," Cockroach Labs Blog, 2023. [Online]. Available: https://www.cockroachlabs.com/blog/living-without-atomic-clocks/

[7] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM Symp. Operating Systems Principles (SOSP)*, Stevenson, WA, Oct. 2007, pp. 205–220.